

OOP Project Requirements (2026 Spring)

Objective:

This is an **individual** project. You must work on this project on your own. The goal of this project is to design and implement a system using Java, with Java Swing for the graphical user interface (GUI). The system will be based on a real-world scenario. Select **one** of the five topics listed below. The project should demonstrate your understanding of core Java concepts, including object-oriented programming (OOP), file handling, and data structures.

Topics:

1. Restaurant Kitchen Order Management System: A system for managing food orders in a restaurant kitchen. Chefs can view incoming orders, mark dishes as in-progress or completed, and track preparation times. Each order contains multiple menu items, and the system automatically updates the status of each dish. Users can add new menu items with descriptions, prices, and estimated preparation times. Advanced features can be sales analytics that display total revenue for the current day, monthly sales breakdown per menu item, and the ability to generate reports showing how many times each dish has been sold within a selected date range.

2. Adventure Quest Game with Reward System: A role-playing style game where players create a character with customizable attributes such as name, class (warrior, mage, archer), and starting stats. Players take on challenges, defeat enemies, and earn rewards including experience points, gold coins, and collectible items. Completing challenges allows characters to level up and improve their abilities. Players can view their inventory, track their progress, and unlock achievements. Advanced features can include a shop system where players can purchase items using earned gold, and a quest log that tracks completed versus pending challenges with rewards displayed for each.

3. Library Study Room Seat Reservation System: An application for students to view real-time seat availability across different library floors and zones (quiet study, group discussion). Users can reserve, check in to, and release seats through a visual seat map. The system includes a credit score mechanism to discourage no-shows, with administrator privileges for monitoring occupancy rates and managing reservations. Advanced features can include usage statistics that show the most popular seats, peak reservation hours, and a waiting list feature that automatically assigns seats when they become available.

4. Music Playlist Manager with Player: A system for users to build and organize their personal music library. You may design features such as adding songs with details like title, artist, album, genre, duration, and file path; creating, editing, and deleting custom playlists; implementing a built-in player with playback controls including play, pause, skip, shuffle, and repeat; searching and filtering the library by various criteria. Advanced features can be playback history tracking, listening statistics, favorites, or other creative enhancements of your choice.

5. Community Pet Adoption Center System: A platform connecting animal shelters with potential adopters. Shelter staff can add, update, and remove pet listings with details such as species, age, size, and health status. Adopters can search and filter available pets, submit adoption applications,

and track application status. Staff can review pending applications and approve or deny requests, updating pet availability accordingly. Advanced features can include a favorite list where adopters can save pets they are interested in, and adoption statistics that display the number of successful adoptions per month and the most popular species adopted.

Based on the topic you selected, you need to **design** your system to provide meaningful features of the system. Any features not mentioned above are welcome, as long as they represent valid scenarios.

Requirements:

1. System Scope

Your system should model a real-world scenario. The system should allow users to perform CRUD operations (Create, Read, Update, Delete) on data entities (e.g., customers, products, transactions, etc.).

2. OOP Concepts

- Use at least one interface to define a set of methods that must be implemented by multiple classes.
- Use at least one abstract class to provide common functionality for related classes.
- Demonstrate inheritance by creating a hierarchy of classes (e.g., a base class and at least two subclasses).
- Use at least 6 **interconnected** classes (**exception classes and GUI classes are not included**).
- Use method overloading.
- Use method overriding.
- Use polymorphism to allow objects of different classes to be treated as objects of a common superclass.
- Use Exception handling to handle error messages. There must be at least two user defined exception classes.

Interconnected classes mean one class is connected to another class in some way. The following relationships all count as interconnected:

- Inheritance – A subclass extends a superclass. For example, Car extends Vehicle.
- Association – A class has a field of another class type. For example, an Order class has a Customer field.
- Aggregation / Composition – A class contains a collection of another class. For example, a ShoppingCart contains a list of Product objects.
- Dependency – A method of one class uses an object of another class as a parameter or local variable. For example, processPayment (Payment p).

3. Data Storage

- Use file I/O to store and retrieve data persistently (e.g., save customer details, transaction history, or inventory to a file).
- The system should load data from a file when the program starts and save data back to the file when the program exits.

- You may also choose to use a database to store your data. No extra credit is provided in this case.

4. Data Structures

Use ArrayList or other Java Collections Framework classes to manage collections of objects (e.g., a list of customers, products, or orders). Therefore, a total amount of orders/deposits can be shown as an output on GUI.

5. GUI Requirements

Implement a Java Swing GUI with a main window that allows users to interact with the system.

The main window should include:

- Input fields (e.g., text fields, dropdowns, buttons) for users to perform CRUD operations.
- A display area (e.g., a JTextArea, JTable, or JList) to show output (e.g., error messages, lists of customers, transactions, or inventory, etc.).
- **All primary functions should be operated within the main window.** Temporary windows (e.g., dialog boxes) may be used for user input when necessary, but they must close immediately after serving their purpose.
- Ensure the GUI is user-friendly and intuitive (e.g., include labels, clear layouts, and error messages for invalid input).

6. Make your project more attractive

- Implement sorting or searching functionality (e.g., sort customers by name, search for a product by ID).
- Add features to do data analysis according to the data the system collects.

Submission (**All on iSpace**):

1. Proposal (**5% due date May 7th 11:59pm**):

Write a short proposal in a **PDF** file (2-3 pages) explaining:

- The real-world scenario you chose and the business logic for the scenario.
- Explain your design of the required OOP concepts (interface, abstract class, inheritance, etc.). You may use UML diagram or detailed explanation of your design.
- Draw your GUI window interface, and explain the features involved in the system.
- Explain how the IO is designed.

Note:

- (1) The proposal accounts for 5% of your total project grade.
- (2) Please describe your design in considerable detail, including the system overview, main features, key classes, operation logic, GUI design, IO design, etc.
- (3) For the "IO design," you may specify what data need to be stored without explaining how they will be stored.
- (4) It is acceptable if your final implementation differs somewhat from your original proposal.
- (5) The deadline for the proposal is **May 7th 11:59pm**. There is no second chance to submit after the deadline.

2. **Source Code (due date May 27th 11:59pm):**

Submit all Java source files (.java) with proper comments for key logic, classes, and methods.

3. **Demo Video (5% due date May 27th 11:59pm):**

Record a video (**less than 5 minutes**) to demonstrate your system. While demonstrating your system, you must use your voice to explain the steps you are showing. We will check if you are demonstrating it yourself. Plagiarism will be considered if we find the person demonstrating your system is not yourself.

Note:

- (1) The purpose of the video is to demonstrate how your application operates.
- (2) You may speak in either English or Chinese (it is the **ONLY** part of the project where you can use Chinese).
- (3) To identify yourself, you must **show your face** and give a brief self-introduction (**about 10 seconds**) at the beginning of the video.
- (4) You do not need to show your face for the entire duration of the video.
- (5) If you make separate videos for your demo and your introduction, make sure we are able to identify it is you talking during the demo.